第 15 章　元学习

15.1　元学习的概念

我们这一章介绍**元学习（meta learning）**。元学习从字面的意思就是“学习”的“学习”，也就是学习如何学习。大部分的深度学习就是在不断的调整超参数，或者在决定网络架构，改变学习率等等。实际上没有什么好方法来调这些超参，今天工业界最常拿来解决调整超参数的方法是买很多张 GPU，然后一次训练多个模型，有的训练不起来、训练效果比较差的话就输入掉，最后只看那些可以训练的比较好的模型会得到什么样的性能。所以在业界做实验的时候往往就是一次开几张 GPU，这些 GPU 跑多组不同的超参数，看看哪一组超参数可以得到最好的结果。但是在学术界我们通常没有那么多张 GPU，通常需要凭着经验和直觉定义可能效果比较好的超参数，然后看看这些超参数会不会得到好的结果。但是这样的方法往往会花费很多时间，因为需要不断的去调整这些超参数。所以我们就会想办法让机器自己去调整这些超参数，机器自己学习一个最优的模型和网络架构，然后得到好的结果。元学习就这样诞生了。

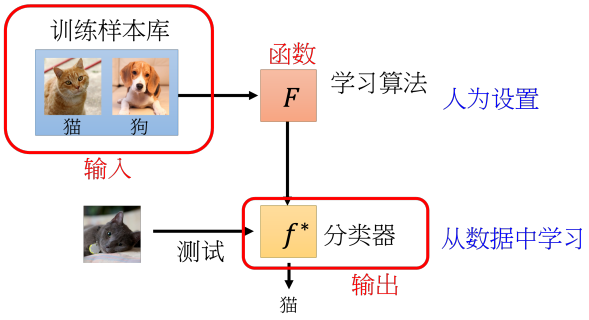


图 15.1　元学习的概念

我们接下来分析元学习的本质以及其主要的三个步骤。首先元学习算法，把它简化来看其实就是一个函数。这个函数我们用 F 来表示它，不同于普通的机器学习算法的输入一张图片，元学习的函数 F 是一个数据集，这个数据集里面有很多的训练数据。我们把训练数据集输入到这个函数 F 里面，它会输出训练完的结果。假设我们要训练的是一个分类器，那这个函数就是输入训练数据，输出就是分类器。有了这个分类以后，我们就可以把测试数据输入，它输出来的结果是我们要的分类结果。所以一个学习算法它是一个函数，我们用 F 来表示它，而它的输入就是训练数据，输出是另外一个函数，我们用 f 来表示它。这个 f 的输入是一张图片，输出是分类的结果，整个元学习的框架如图 15.1 所示。所以元学习的目标就是要找到一个 F，这个 F 可以让 f 的损失越小越好。那这个 F 函数是人为设定的，或者说是我们提前设置的。我们其实也可以直接学习这个 F 函数，对应于我们在机器学习里面介绍的三个步骤，在元学习里，其实我们要找的也是一个函数，只是这个函数跟机器学习一般要找函数不一样。在元学习里我们要找的函数是一个学习算法。我们下面分别类比机器学习的三个步骤来介绍元学习中的三个步骤，来寻找学习函数。

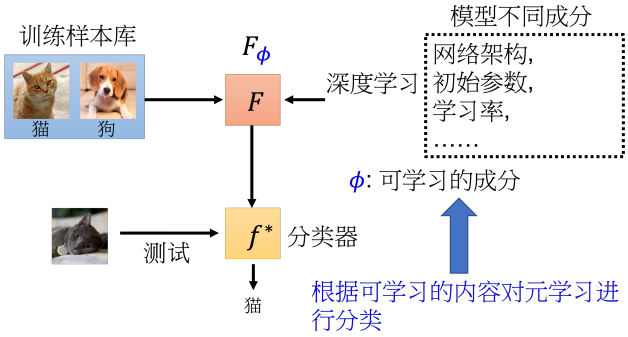


图 15.2　元学习的步骤一：学习算法

15.2　元学习的三个步骤

首先第一个步骤（如图 15.2 所示）是我们的学习算法里要有一些要被学的东西，就像在机器学习里面神经元的权重和偏置是要被学出来的一样。在元学习里面，我们通常会考虑要让机器自己学习网络的架构，让机器自己学习初始化的参数，让机器自己决定学习率等等，我们期待它们是可以通过学习算法被学出来的，而不是像机器学习一样我们人为设定的。我们把这些在学习算法里面想要它自学的东西统称为 ϕ，在机器学习里面我们是用来 θ 代表一个函数里面我们要学的东西。接下来，我们都将学习算法写为 F_ϕ，代表这个学习算法里面有些是未知的参数。对于不同的元学习的方法，它会想办法去学模型不同的成分，当去学不同模型成分的时候，我们就有了不同的元学习的方法。

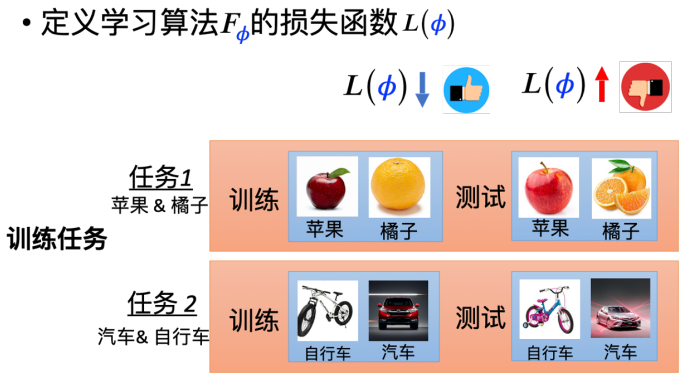


图 15.3　元学习的步骤二：定义损失函数

接下来，第二步（如图 15.3 所示）是设定一个损失函数，损失函数在元学习里是要决定学习算法的好坏。$L(\phi)$ 代表现在用 ϕ 作为参数的算法的性能。如果 $L(\phi)$ 的值很小就代表它是一个好的学习算法，反之它就是一个不好的学习算法。我们需要如何决定这个 L 损失函数呢？在机器学习里面，损失函数来自于训练数据，那在元学习里面，我们收集的是训练任务。举例来说，假设今天想要训练一个二分类的分类器，来分辨苹果和橘子（任务一），以及分别脚踏车和车（任务二），以上每一个任务里面我们都会有分训练数据和测试数据。

接下来我们就要分析在元学习中的损失函数 L 应该如何定义。我们评价一个学习算法的好坏，是看其在某一个任务里面使用训练数据学习得到的算法的好坏。比如，任务一是分辨苹果和橘子，我们就把任务一里的训练数据拿出来输入给这个学习算法，从而学出一个分类。我

们使用 $f_{\theta^{1*}}$ 来代表这个是任务一的分类，分辨苹果和橘子。如果这个分类是好的，那就代表我们的 L 的规则是好的，反之如果这个分类是不好的，就代表说这个学习算法是不好的。对于不好的学习算法（测试数据中表现不好的），我们就会给它比较大的损失 $L(\phi)$。

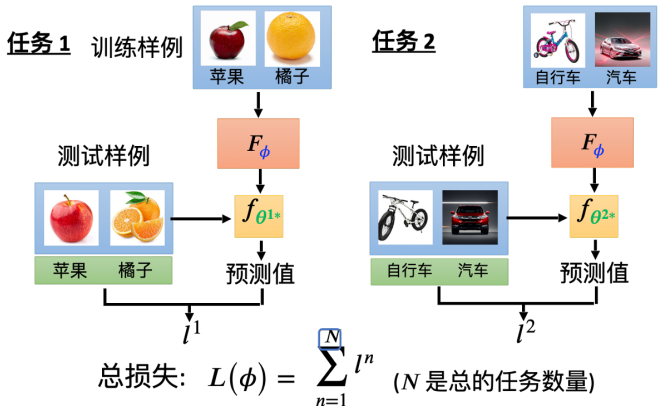


图 15.4　元学习的步骤二中的多任务分类

到目前为止，我们都只考虑了一个任务，那在元学习中我们通常不会只考虑一个任务，也就是我们不会只用苹果和橘子的分类来看一个二分类学习算法的好坏。我们还希望拿别的二元分类的任务来测试它，比如说区分自行车和汽车的训练数据（如图 15.4 所示），输入给这个学习算法，让它进行分类。像这两个学习算法是一样的，但是因为输入的训练数据不一样，所以产生的分类也不一样。θ^{1*} 代表的是这个学习算法在任务一（分类苹果和橘子）上面学习得到的参数，θ^{2*} 代表的是这个学习算法在任务二（分类自行车和汽车）上面学习得到的参数。与任务一相同，任务二它自身也有一些测试的数据，我们一样把它的测试数据输入给 $f_{\theta^{2*}}$，然后看看得到的正确率如何，就可以计算出其在任务二中的表现。所以我们现在知道这个学习算法 $L(\phi)$ 在任务一和任务二上的表现以后，就可以把任务一和任务二上的表现把它加起来，得到这个整个学习算法的损失。当然如果扩展到 N 个任务的话，整个损失就是一个 N 个任务的损失，我们把这个损失写成 $L(\phi)$，这个损失就是我们的元学习的损失。其代表了现在这个学习算法学习所有问题的表现有多好。

大家应该已经关注到了一件事情，元学习中在每一个任务计算损失的时候，我们是用测试数据来进行计算。而在一般的机器学习里面，所谓的损失其实是用训练数据来进行计算的。这个问题是因为我们的训练单位是任务，所以可以用训练的任务里面的测试数据，训练的任务里面的测试数据是可以在元学习的训练的过程中被使用到的。我们将元学习的演算法介绍完以后，再会把元学习和机器学习再做一次比较，届时会更加清楚。

最后，元学习的第三个任务就是要找一个学习算法，即找一个 ϕ 让损失越小越好。这件事怎么做呢？我们已经写出了损失函数 $L(\phi)$，是 N 个任务的损失的总和。我们现在要找一个 ϕ 使得 $L(\phi)$ 最小，我们将这个 ϕ 定义为 ϕ^*。解这个优化问题的过程有很多，比如之前我们介绍过的梯度下降；如果没有办法计算梯度，也可以用强化学习的方法来解这个优化问题，或者使用进化算法来解这个优化问题。总之，我们就可以让机器自己找出来的一个学习算法 F_{ϕ^*}，这个学习算法 F_{ϕ^*} 就是我们的元学习算法。

我们再来回顾一下元学习的三个过程，如图 15.5 所示。首先收集一批训练数据，这些训练数据是由很多个任务组成的，并且每一个任务都有训练数据和测试数据。根据这些训练数据通过我们刚才讲的三个步骤，可以得到一个学习算法 F_{ϕ^*}。接下来我们就可以用这个学习

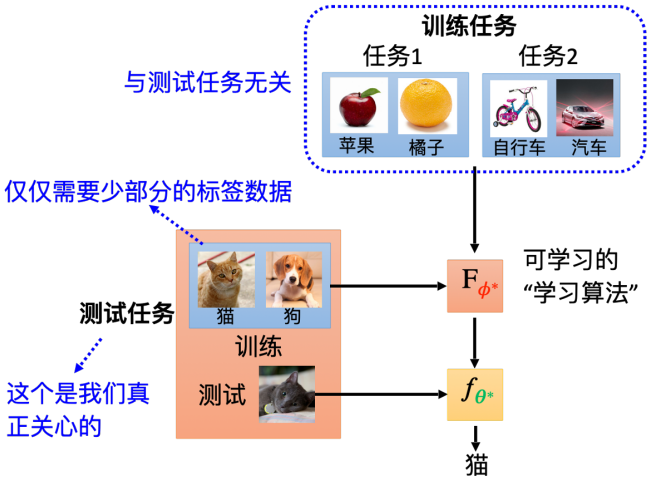


图 15.5　元学习的完整框架

算法 F_{ϕ^*} 来进行测试。假设训练的时候，训练任务是教机器分苹果和橘子以及自行车和汽车，测试的时候是要分猫和狗，那每一个任务里面都有训练数据，也都有测试数据。我们就需要把测试任务里面的训练数据学出一个分类，然后再把这个分类用在测试任务里面的测试数据上。其中我们真正关心的是测试任务里面的测试数据，因为这个测试数据是我们真正要分类的东西。

很多人会觉得小样本学习和元学习非常的像，所以我们也简单区分一下元学习和小样本学习。简单来说，小样本学习指的是期待机器只看几个样例，比如每个类别都只给他三张图片，它就可以学会做分类。而我们想要达到小样本学习中的算法通常就是用元学习得到的学习算法，这就是两者的关系。

15.3　元学习与机器学习

这一小节比较一下机器学习和元学习的差异。首先来看一下机器学习和元学习的目标，如图 15.6 所示。机器学习的目标是要找一个函数 f，这个函数可以是一个分类器，把几百张图片输入进去，分类器就告诉我们分类的结果。元学习一样是找一个函数，但它要找的是一个学习算法 F_{ϕ^*}，这个学习算法可以接受训练数据，然后输出一个分类器 f。这个 F_{ϕ^*} 将训练数据作为输入，它直接输出训练的分类结果 f，这个 f 就是我们想要的分类器。

从训练数据角度分析，在机器学习里面，我们是拿某一个任务里面的训练数据进行训练，而在元学习中我们是拿训练的任务来进行训练。这个很容易搞混，所以在文献中，我们会把任务里面的训练数据叫做支持（support），把测试数据叫做查询（query）。在元学习里面，我们是拿查询来进行训练，而在机器学习里面，我们是拿支持来进行训练。

那在机器学习里面，我们需要手动设置一个学习算法，而在元学习里面，我们是有一系列的训练任务，所以我们也将元学习中的学习算法部分的学习称为跨任务学习。而对应的机器学习中的学习称为单一任务学习，因为我们是在一个任务里面进行学习。

我们再看一下两者的完整框架，如图 15.7 所示。在机器学习中，完整的框架就是把训练数据拿去产生一个分类器，接着再把测试数据输入到这个分类器里面得出分类的结果。而在元学习中，我们是有一些训练的任务，把这些训练的任务拿来产生一个学出来的学习算法叫做 F_{ϕ^*}。对于接下来的测试任务，测试任务里面有支持数据和查询数据，我们再把这个测试任

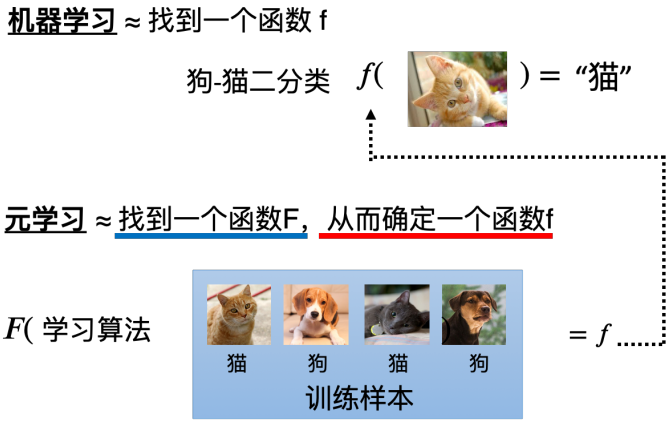


图 15.6　元学习和机器学习的目标

务里面的训练数据输入到学习出来的学习算法里面，得到一个分类器后，再把测试数据输入进去，得到分类的结果。我们把元学习里面的这个测试叫做跨任务测试，因为它不是一般的测试。一般的机器学习，我们的这个测试叫做单一任务测试，因为我们是在一个任务里面进行测试。在元学习里面，我们要测试的不是一个分类表现的好坏，而是一个学习算法的表现的好坏，所以在元学习里面为跨任务的测试。那有时候我们也在一些论文中会看到整个流程中一次单一任务的训练和一次跨任务的测试，我们把这个流程叫做一个回合。所以在元学习里面，我们是在一个回合里面进行训练和测试，而在机器学习里面，我们是在一个任务里面进行训练和测试。

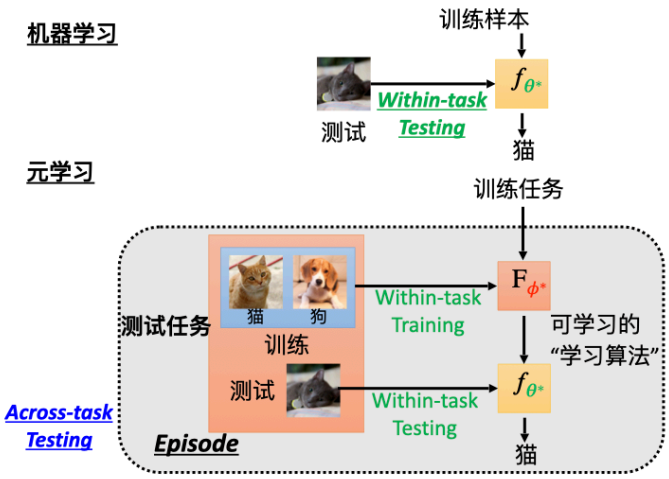


图 15.7　元学习和机器学习的框架对比

对于损失，在机器学习中我们使用 $L(\theta) = \sum_{k=1}^{K} e_k$ 表示损失函数，其中 e_k 表示第 k 个训练样本的损失，其中的加和为所有训练数据在一个任务中的损失总和。在元学习里面，我们使用 $L(\phi) = \sum_{n=1}^{N} l^n$ 表示损失函数，其中的 l^n 表示第 n 个测试样本中的损失，其中的加和为在所有任务中的损失总和。

对于训练的过程两者也有一些差异，元学习的训练需要算 l^n，也就是计算每一个小任务的损失函数，在这个过程中，我们需要做一次单一任务的训练 + 测试，也就是一个回合。现

在假设我们的优化算法中，要找一个 ϕ 让 $L(\phi)$ 最小，那做这件事情的时候，我们需要算这个 L 很多次，也就是跨任务的训练包含了很多次的单一任务的训练 + 测试。这个是非常复杂的且耗时的。所以在文献中，学习如何去初始化整个过程的工作中，往往把跨任务训练叫做外循环，把单一任务训练叫做内循环。这是因为在跨任务训练里要跑好几次单一任务训练，所以跨任务训练是外循环，单一任务训练是内循环。不过，外循环和内循环这些称呼通常只有在学习初始化的工作中才会有，其他时候通常也不会这样叫。

刚才介绍的都是元学习和机器学习的差别，他们其实也有很多的共同之处，事实上很多在机器学习那边学到的知识和基本概念都可以直接搬到元学习来。举例来说，在机器学习上面你会害怕训练数据上可能会有过拟合的问题，那在元学习里面也有可能会有过拟合的问题，比如机器学习到了一个学习算法，这个学习算法在训练任务上做得很好，面对一个新的测试的任务反而会做得不好，所以元学习也有可能有过拟合的问题。如果遇到过拟合问题应该怎么办呢？我们类比一下机器学习，在机器学习里面，最直观的方法就是收集更多的训练数据，所以在元学习里面也可以做一样的事——收集更多的训练任务。也就是如果训练的任务越多，就代表训练的数据样本越多，那学习算法就越有机会可以泛化并用到新的任务上面。

另外，我们在机器学习上会做数据增强，也就是在训练的时候，我们会把训练数据做一些变化，比如说把图片做一些旋转、平移、缩放等等，这样可以让训练数据更多。在元学习里面我们同样也可以做数据增强，也可以想一些方法来增加训练任务，比如说我们可以把训练任务做一些变化，比如说把训练任务的类别做一些变化，或者把训练任务的数据做一些变化等等。此外，我们在做元学习的时候还是要做优化，我们还是要想办法去找一个 ϕ 让 $l(\phi)$ 越小越好。假设我们最小化 $l(\phi)$ 的方法是梯度下降法，那我们在做梯度下降法的时候，我们还是要去调学习率，只不过与机器学习不同，我们需要调整的参数是可学习的学习算法的参数。有人可能会问，既然都要调整参数，那我何必还要用元学习，直接讲每一个机器学习的问题调整参数不就可以了吗？其实并不是，因为在元学习里面，我们只需要把学习算法的参数调整好，其就可以一劳永逸的使用到其他的任务当中，而不需要每一个任务都去调整参数。这样的话，我们就可以节省很多的时间，也可以让我们的学习算法更加的有效率。

那既然说到要调参数就遇到另一个问题了，在机器学习中我们不仅仅有训练样本和测试样本，同时还有验证集的样本，用验证集样本中的表现来选择你的模型。所以元学习中也应该要有用于验证的任务，也就是说在元学习中，我们应该要有训练任务、验证任务和测试任务。其中验证任务来决定训练学习算法的时候的一些超参数，然后才跑在测试的任务中。

15.4 元学习的实例算法

前面我们已经讲完了元学习的基本概念，接下来我们就要讲一些元学习的实例算法。在这里我们会介绍两个算法，一个是**模型诊断元学习(model-agnostic meta-learning，MAML)**，另一个是 Reptile。这两个算法都是在 2017 年提出来的，而且都是基于梯度下降法进行优化的。那我们最常用的学习算法是梯度下降，在梯度下降中，我们要有一个网络架构，同时初始化一下这个网络的参数 θ^0。我们也有一些训练数据，他们是从训练数据里面采样一个批次出来，然后用这个批次来计算梯度，并用这个梯度来更新参数，从 θ^0 到 θ‘，接着再重新计算一次梯度，再更新参数，就反复这样下去，直到次数够多，输出一个满意的 θ^* 出来。

初始化的参数 θ^0 可以训练的，一般 θ^0 是随机初始化的，也就是从某一个固定的分布里面采样出来的。同时 θ^0 对结果往往有一定程度的影响，好的初始化参数不好的初始化参数可以天差地远，那我们能不能够透过一些训练的任务，来找出一个对训练特别有帮助的初始化

参数呢？这个方法是模型诊断元学习。

MAML 的基本思路是，算法要最大化模型对超参数的敏感性。也就是说，要让学习到的超参数让模型的损失函数因为样本的微小变化而有较大的优化。因此，模型的超参数设置应该能够让损失函数变化的速度最快，即损失函数此时有最大的梯度。因此，损失函数被定义为每一个任务下该模型的损失函数的梯度的和。剩下要做的，就是根据这个定义的损失函数，用梯度下降法进行求解。在训练的过程中算法会求取以下两次梯度。第一次求梯度：针对每个任务，计算损失函数的梯度，进行梯度下降；第二次求梯度：对梯度下降后的参数求和，再求梯度，进行梯度下降。当然，MAML 有另外的变形就叫做 Reptile，翻译过来叫爬虫，大家可以自行了解。需要补充的是，虽然在 MAML 中，我们要去学习初始化参数的过程，但是在其中我们也是有很多超参数需要自己决定的。

这里做一个联想，我们在介绍自监督学习的时候，我们也有提到好的初始化参数的重要性。在自监督学习中，我们就是有很多的没有标记的数据，可以用一些代理的任务去训练它，比如说在 Bert 里面就是用填空题来训练模型，在图像上也可以做自监督学习。比如把图片的其中一块盖起来，让机器预测被盖起来的一块是什么东西，机器就可以从中学到一些特征，然后再把这些特征用在其他的任务上。当然，现在在做图像的自监督学习的时候，可能这个掩码的方法以及所谓的填空的方法不是最常用的，目前比较流行用对比学习的方法。总之，在自监督学习中我们会先拿一大些的数据去做预训练，那预训练的结果我们也说它是好的初始化参数，然后再把这些好的初始化参数用在测试的任务上。

那这 MAML 和自监督学习有什么不同呢？其实它们的目的都是一样的，都是要找到好的初始化参数，但是它们的方法不一样。自监督学习是用一大些的数据去做预训练，而 MAML 是用一大些的任务去做预训练。另外过去在自监督学习还没有兴起的时候，也有一些方法是用一大些的任务去做预训练，这个方法叫做多任务学习。具体来讲，我们一样有好几个任务的数据，并且把这些好几个任务的数据通通放在一起，然后接下来我们同样可以找到一个好的初始化参数，然后再把这个好的初始化参数用在测试的任务上，这就是多任务学习。现在我们在做有关 MAML 研究的时候，通常会把这种多任务学习的训练方法来当做元学习的基线。因为这两个方法他们用的数据都是一样的，一边只是我们会把不同的任务分开，另外一边把所有的任务的数据倒在一起。

其实 MAML 很像是域适应或者迁移学习，也就是我们在某些任务上面学到的东西可以被被迁移到另外一个域，我们可以说他们是基于分类问题的域适应或者迁移学习。所以我们在研读文献的时候其实也不用太拘泥于这些词汇，我们要真正要在意的是这些词汇背后所代表的含义是什么。

我们下面解释一下 MAML 的优势。首先有两个假设，第一个假设是 MAML 找到的初始参数是一个很厉害的初始参数。它可以让我们的例如梯度下降这种学习算法快速找到每一个任务的参数。另外一个假设是这个初始化的参数它本来就和每一个任务上最终好的结果已经非常接近了，所以我们直接应用很少几次的梯度下降就可以轻易的找到好的结果，这个也是使得 MAML 有效的关键。当然 MAML 也有一些变形，比如 ANIL（almost no inner loop）、First order MAML（FOMAML）以及 Reptile 等等，这里我们不做扩展。

除了可以学习初始化的参数外，MAML 还可以学习优化器，如图 15.8 所示。我们在更新参数的时候，需要决定比如说学习率、动量等等的参数。像学习率这种超参数进行自动更新的方法在很早以前就有了，NIPS2016 年就有一篇文章，叫做“Learning to learn by gradient descent by gradient descent”。在这篇文章里面呢，作者直接学习了优化器，一般我们的优化

器都是人为规定的（比如 ADAM 等等），而这个文章中的参数是根据训练的任务自动学出来的。

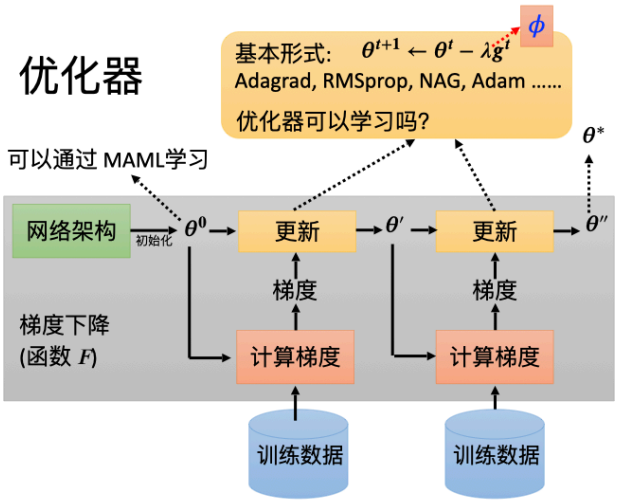


图 15.8　MAML 中可学习的优化器

当然我们还可以训练网络架构，这部分的研究被称为**神经网络架构搜索 (Neural Architecture Search，NAS)**。如果在元学习中我们学习的是网络的架构，讲网络架构当作 ϕ 的话，那我们就是在做 NAS。在 NAS 里面，ϕ 是网络架构，我们要找一个 ϕ 去最小化 $L(\phi)$。这其中我们做 $L(\phi)$ 的微分就有可能出问题了。当我们遇到优化问题时并且没办法上微分的时候，强化学习也许是一个解决方法。具体做法是，我们可以把 ϕ 想成是一个智能体的参数，这个智能体的输出是 NAS 中相关的超参数。举例来说，现在第一层的滤波器它的长是多少，宽是多少，步长是多少，数目是多少等等。我们智能体的输出就是 NAS 中相关的参数，然后接下来我们要训练智能体，让它去最大化一个回报，即 $-L(\phi)$，让他去最大化这个回报，就等于最小化 $L(\phi)$，也就是我们要找的最优的 ϕ。

下面是从一篇文章中截取的一个 NAS 的例子，以此为例介绍 NAS 的过程，如图 15.9 所示。具体来讲，我们有一个智能体是 RNN 架构。这个 RNN 架构每次会输出一个网络架构有关的参数，比如它会输出滤波器的高是多少，再输出过滤器的宽是多少，接着再输出步长是多少等等。第一层第二层输出完了以后，接下来输出 n+1 层，再输出 n+2 层，以此类推。有了这些参数以后，就根据它们设计一个网络以后就去训练这个网络，这个过程其实就是之前我们介绍的单一任务训练。然后接下来就去做强化学习，我们可以把这一个网络他在测试数据上面的准确率当做回报来训练智能体，目标是最大化回报，这个过程其实就是跨任务训练。那除了强化学习以外，其实用演变算法也是可以的。本质上其实就是硬要把网络架构改一下，让它变得可以微分。其中有一个经典的做法叫做可微分架构搜索，这个方法的本质就是想办法让问题变的是可以微分，从而可以直接用梯度下降来最小化这个损失函数。

除了网络架构可以学习外，其实数据处理部分也有可能可以学习。我们在训练网络的时候，通常要做数据增强。那在元学习中我们可以让机器自动进行数据增强。另一个角度，我们在训练的时候，有时候会需要给不同样本不同的权重。具体操作的话就会有不同的策略，比如有的策略就是如果有一些样例距离分类边界线特别近，那说明其很难被分类，这样类似的样例也许就要给它们比较大的权重，这样网络就会聚焦在这些比较难分别的样例中，希望它们

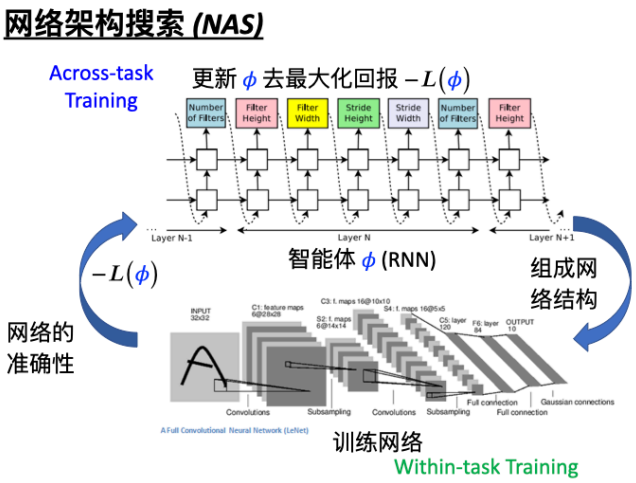


图 15.9　NAS 的实例

可以被学得比较好。但是也有文献有相反的结论，比如比较有干扰噪声的这些样本应该给它比较小的权重，这些样例如果比较接近分类边界线，可能代表它比较有噪声干扰，代表它可能标签本身就标错了，或者分类是不合理的等等，也许就应该给这些样本比较小的权重。那元学习中如何决定这个采样权重的策略呢？我们可以用学习的方式把采样策略直接学出来，然后根据采样数据的特性自动决定采样数据的权重如何设计。

到目前为止，我们看到的这些方法都是基于梯度下降再去做改进的，我们有没有可能完全舍弃掉梯度下降呢？比如，我们有没有可能直接学习一个网络，这个网络的参数 ϕ，网络直接训练数据作为输入直接输出训练好的结果？如果真的有这样一个网络，它可以将训练数据作为输入输出训练好的网络的参数，那我们就可以让机器发明了新的学习算法。这个是有可能的，并且已经有了一些论文。不过到目前为止，我们还是把训练跟测试分成两个阶段，我们有一个学习算法使用训练数据进行训练，然后输出训练好的结果，并把训练好的结果用在测试数据上。我们想看看有没有可能更进一步，直接把整个回合，也就是一次训练加一次测试合并在一起。有一些工作，它就是直接把训练数据和测试数据当做网络的输入，网络输入完训练数据以后，机器也许学出了一个学习算法，也许找出一组参数等等，再它输入训练数据以后再给他测试数据，它就可以直接输出这些测试数据的答案。这个时候我们不再有训练和测试的分界，在一个回合里面不再分训练和测试，而是直接用个网络把训练和测试这件事情一次搞定。目前这种系列的方法叫做“learning to compare”，它又叫做基于度量的方法，这一系列的做法就可以看作是训练和测试没有分界，一个网络直接把训练数据和测试数据都读进去，而直接输出测试数据的结果。

15.5　元学习的应用

本章最后，我们再简单分享一些元学习的应用。现在在做元学习的时候，我们最常拿来测试元学习技术的任务叫做少样本图像分类，简单来讲就是每一个任务都只有几张图片，每一个类别只有几张图片。比如我们使用图 15.10 的案例为例说明。现在分类的任务是分为三个类别，每个类别都只有两张图片作为输入，我们希望通过这样一点点的数据就可以训练出一个模型。也就是给这个模型一张新的图片，它可以知道这张图片属于哪一个类别。在做这种

少样本图像分类的时候，我们会常看到一个名词叫做 N 类别 K 样例下的分类任务，这个名词是什么意思呢？N 类别 K 样例的分类任务，它的意思就是在每一个任务里面，我们有 N 个类别，而每一个类别我们只有 K 个样例。举例来说，在图 15.10 这个例子里面我们有三个类别，每一个类别只有两个样例，那它就是 3 类别 2 样例分类。在元学习里，如果我们要教机器能够做 N 类别 K 样例分类，那意味着说我们需要准备很多的 N 类别 K 样例下的分类任务当做训练的任务，这样机器才能够学到 N 类别 K 样例的算法。



图 15.10　少样本的案例分析

那要怎么去找一系列的 N 类别 K 样例下的任务呢？在文章中最常见的一种做法是使用 Omniglot 当做基准，Omniglot 是一个手写的数据集，它有 1623 个不同的字符，每一个字符有 20 个样例。那有这些字符以后呢，我们就可以去制造 N 类别 K 样例下的分类。比如我们从 Omniglot 里面选出 20 个字符，然后每一个字符就只取一个样例，这样就得到一个 20 类别 1 样例的分类任务。如果我们把这个任务当做训练数据，那我们就可以让机器学习到 20 类别 1 样例的分类算法。如果我们把这个任务当做测试数据，那我们就可以测试机器在 20 类别 1 样例的分类任务上的表现。同理，我们可以制造出 20 类别 5 样例的分类任务，这个任务里面每一个类别都有 5 个样例，然后我们可以把这个任务当做训练数据，让机器学习到 20 类别 5 样例的分类算法。

在使用 Omniglot 的时候，我们会把字符分成两半，一半是拿来制造训练任务的字符，另外一半是拿来制造测试任务的字符。如果我们要去制造一个 N 类别 K 样例的任务，那么就是从这些训练任务的字符里面先随机采样 N 个字符，然后这 N 个字符每个字符再去采样 K 个样例，集合起来就得到一个训练的任务。对于测试的任务，就从这些测试的字符里面拿出 N 个字符，然后每个字符采样 K 个样例，你就得到一个 N 类别 K 样例下的测试任务。这样我们就可以把 Omniglot 当做一个基准，然后在这个基准上面测试不同的元学习算法。

总之，元学习不是只能用非常简单的任务，今天在学界已经开始把元学习推向更复杂的任务，我们也一直希望未来元学习这个技术能够真的用在现实的应用上，可以走得多远好。